

Architecture - Assistant intelligent HEL

PETITJEAN Gabriel

May 11, 2026

Table des matières

1	Architecture de l’assistant intelligent	3
1.1	Configuration du modèle	3
1.2	Flux de travail de l’Orchestrateur	3
1.3	Récupération hybride (RAG)	5
1.4	Gestion de la mémoire et du cache	5

1 Architecture de l’assistant intelligent

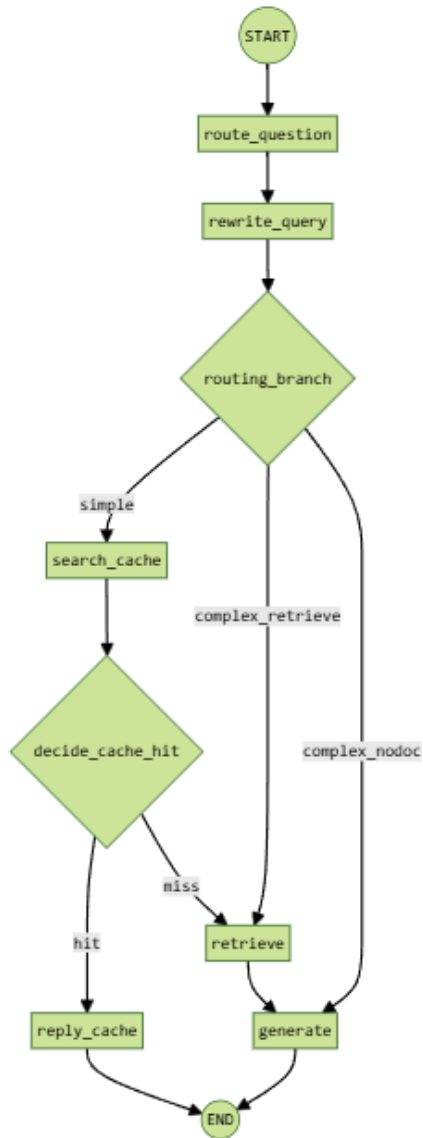
1.1 Configuration du modèle

Le modèle utilisé est *Llama 3.3-70b* via l’API Groq (*ChatGroq*). Ce choix permet d’obtenir des meilleurs performances comparé à un modèle local sur mon ordinateur. La température est fixée à 1 pour permettre une certaine souplesse dans la formulation, tandis que la précision factuelle est assurée par le système RAG et des consignes système strictes.

```
self.model = ChatGroq(model=model_name, temperature=1)
```

1.2 Flux de travail de l’Orchestrateur

L’implémentation de l’orchestrateur repose sur *LangGraph*, qui maintient un *StateGraph* orchestrant le flux de traitement des messages.



Le flux est composé des nœuds suivants :

route_question : Utilise un *SemanticRouter* (avec *HuggingFaceEncoder*) pour classifier la question (FAQ simple vs requête complexe) et décide si une recherche documentaire est nécessaire.

rewrite_query : Reformule la question de l'utilisateur en une requête autonome et compréhensible sans le contexte de l'historique de la conversation. Ce nœud est crucial pour traiter les questions de suivi (ex : "Et leur

adresse?”).

search_cache : Interroge un gestionnaire de cache (*CacheManager*) pour voir si une réponse identique ou très similaire a déjà été validée.

retrieve : En cas de ”cache miss”, ce nœud effectue une recherche hybride via un *EnsembleRetriever* combinant une recherche sémantique (vecteurs) et une recherche par mots-clés (BM25).

generate : Produit la réponse finale en intégrant le résumé global et les documents spécifiques récupérés, tout en respectant des consignes de sécurité (interdiction de ”halluciner” ou de guider l’utilisateur vers l’extérieur).

reply_cache : Renvoie directement la réponse trouvée dans le cache pour minimiser la latence et les coûts.

1.3 Récupération hybride (RAG)

L’assistant utilise une approche d’ensemble pour la récupération :

- **BM25Retriever** : Efficace pour les termes techniques exacts.
- **VectorRetriever** : Capte le sens sémantique de la question.

Les résultats sont pondérés (30% BM25, 70% Vecteurs) pour garantir la meilleure pertinence possible.

1.4 Gestion de la mémoire et du cache

La persistance de la conversation est assurée par un *MemorySaver* avec gestion de sessions (*thread_id*). De plus, un système de cache persistant (SQLite) est utilisé pour stocker les réponses aux questions fréquentes, améliorant ainsi le temps de réponse pour les requêtes récurrentes.